

LAB – 2

Search Algorithm Comparison:

Code :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
int main() {
```

```
    int a[1000], key, i, j, temp;
```

```
    int low, high, mid, index;
```

```
    clock_t start, end;
```

```
    double linear_time, binary_time;
```

```
    srand(time(0));
```

```
    for(i = 0; i < 1000; i++)
```

```
        a[i] = rand() % 10000;
```

```
    printf("Enter element to search: ");
```

```
    scanf("%d", &key);
```

```
    // Linear Search
```

```
    start = clock();
```

```
    for(j = 0; j < 100000; j++) {
```

```
        index = -1;
```

```
for(i = 0; i < 1000; i++) {  
    if(a[i] == key) {  
        index = i;  
        break;  
    }  
}  
}
```

```
end = clock();
```

```
linear_time = (double)(end - start) / CLOCKS_PER_SEC;
```

```
if(index != -1)  
    printf("Linear Search: Found at index %d\n", index);  
else  
    printf("Linear Search: Not found\n");
```

```
printf("Linear Search Time = %f seconds\n\n", linear_time);
```

```
// Sorting for Binary Search
```

```
for(i = 0; i < 999; i++) {  
    for(j = i + 1; j < 1000; j++) {  
        if(a[i] > a[j]) {  
            temp = a[i];  
            a[i] = a[j];  
            a[j] = temp;
```

```
    }  
  }  
}
```

```
// Binary Search
```

```
start = clock();
```

```
for(j = 0; j < 1000000; j++) {
```

```
    low = 0;
```

```
    high = 999;
```

```
    index = -1;
```

```
    while(low <= high) {
```

```
        mid = (low + high) / 2;
```

```
        if(a[mid] == key) {
```

```
            index = mid;
```

```
            break;
```

```
        }
```

```
        else if(a[mid] < key)
```

```
            low = mid + 1;
```

```
        else
```

```
            high = mid - 1;
```

```
    }
```

```
}
```

```
end = clock();
```

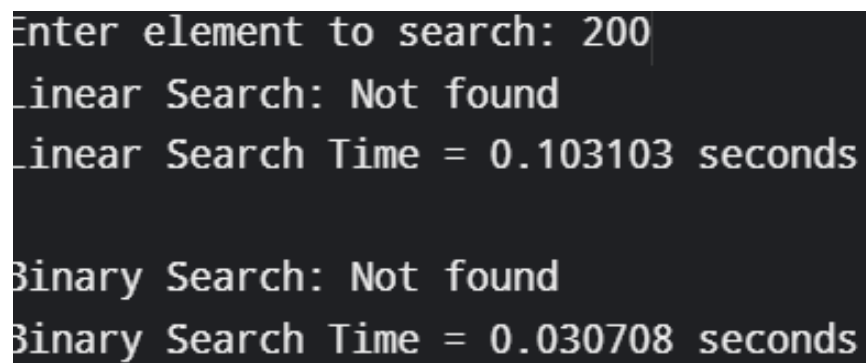
```
binary_time = (double)(end - start) / CLOCKS_PER_SEC;

if(index != -1)
    printf("Binary Search: Found at index %d\n", index);
else
    printf("Binary Search: Not found\n");

printf("Binary Search Time = %f seconds\n", binary_time);

return 0;
}
```

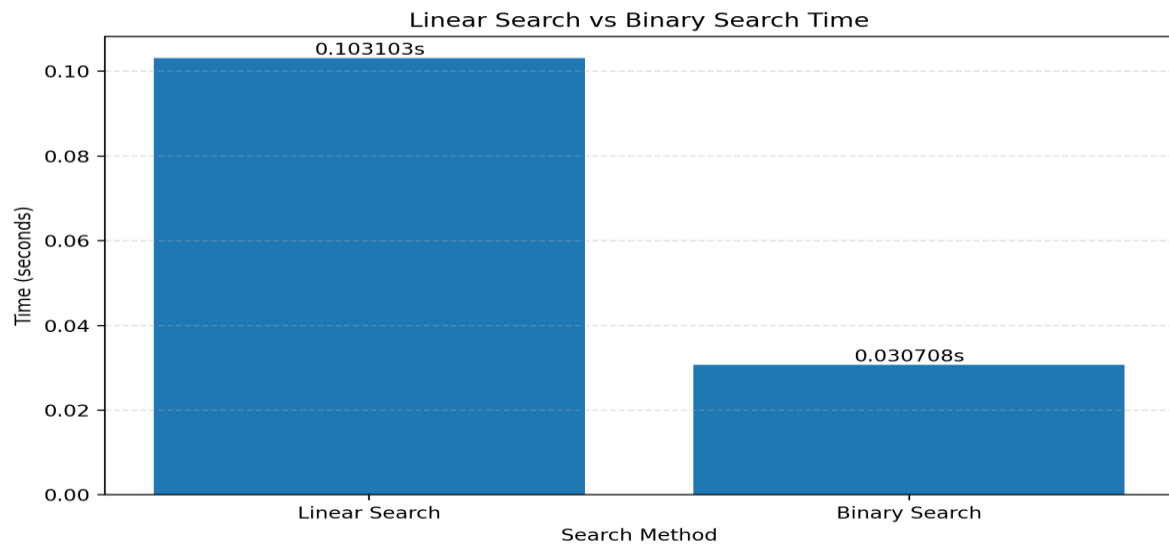
Output



```
Enter element to search: 200
Linear Search: Not found
Linear Search Time = 0.103103 seconds

Binary Search: Not found
Binary Search Time = 0.030708 seconds
```

Bar Chart



Sorting Algorithm Comparison:

Code :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <time.h>
```

```
void bubbleSort(int a[], int n) {
```

```
    int i, j, temp;
```

```
    for(i = 0; i < n - 1; i++) {
```

```
        for(j = 0; j < n - i - 1; j++) {
```

```
            if(a[j] > a[j + 1]) {
```

```
                temp = a[j];
```

```
                a[j] = a[j + 1];
```

```
                a[j + 1] = temp;
```

```
    }  
  }  
}
```

```
void selectionSort(int a[], int n) {
```

```
    int i, j, min, temp;
```

```
    for(i = 0; i < n - 1; i++) {
```

```
        min = i;
```

```
        for(j = i + 1; j < n; j++) {
```

```
            if(a[j] < a[min])
```

```
                min = j;
```

```
        }
```

```
        temp = a[i];
```

```
        a[i] = a[min];
```

```
        a[min] = temp;
```

```
    }
```

```
}
```

```
void insertionSort(int a[], int n) {
```

```
    int i, j, temp;
```

```
    for(i = 1; i < n; i++) {
```

```
        temp = a[i];
```

```
        j = i - 1;
```

```

    while(j >= 0 && a[j] > temp) {
        a[j + 1] = a[j];
        j--;
    }

    a[j + 1] = temp;
}

void merge(int a[], int low, int mid, int high) {
    int temp[1000];
    int i = low, j = mid + 1, k = low;

    while(i <= mid && j <= high) {
        if(a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= mid)
        temp[k++] = a[i++];

    while(j <= high)
        temp[k++] = a[j++];

    for(i = low; i <= high; i++)
        a[i] = temp[i];
}

```

```

void mergeSort(int a[], int low, int high) {
    int mid;

    if(low < high) {
        mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

```

```

void quickSort(int a[], int low, int high) {
    int i, j, pivot, temp;

    if(low < high) {
        pivot = a[high];
        i = low - 1;

        for(j = low; j < high; j++) {
            if(a[j] < pivot) {
                i++;

                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

```

```
}
```

```
temp = a[i + 1];
```

```
a[i + 1] = a[high];
```

```
a[high] = temp;
```

```
quickSort(a, low, i);
```

```
quickSort(a, i + 2, high);
```

```
}
```

```
}
```

```
int main() {
```

```
int original[1000], a[1000];
```

```
int i, j;
```

```
clock_t start, end;
```

```
double bubble_time;
```

```
double selection_time;
```

```
double insertion_time;
```

```
double merge_time;
```

```
double quick_time;
```

```
srand(time(0));
```

```
for(i = 0; i < 1000; i++)
```

```
original[i] = rand() % 10000;
```

```
start = clock();
```

```
for(j = 0; j < 100; j++) {  
    for(i = 0; i < 1000; i++)  
        a[i] = original[i];  
  
    bubbleSort(a, 1000);  
}
```

```
end = clock();
```

```
bubble_time = (double)(end - start) / CLOCKS_PER_SEC;
```

```
start = clock();
```

```
for(j = 0; j < 100; j++) {  
    for(i = 0; i < 1000; i++)  
        a[i] = original[i];  
  
    selectionSort(a, 1000);  
}
```

```
end = clock();
```

```
selection_time = (double)(end - start) / CLOCKS_PER_SEC;
```

```
start = clock();
```

```
for(j = 0; j < 100; j++) {  
    for(i = 0; i < 1000; i++)  
        a[i] = original[i];
```

```
    insertionSort(a, 1000);  
}
```

```
end = clock();
```

```
insertion_time = (double)(end - start) / CLOCKS_PER_SEC;
```

```
start = clock();
```

```
for(j = 0; j < 100; j++) {  
    for(i = 0; i < 1000; i++)  
        a[i] = original[i];
```

```
    mergeSort(a, 0, 999);  
}
```

```
end = clock();
```

```
merge_time = (double)(end - start) / CLOCKS_PER_SEC;
```

```
start = clock();
```

```
for(j = 0; j < 100; j++) {
```

```

    for(i = 0; i < 1000; i++)
        a[i] = original[i];

    quickSort(a, 0, 999);
}

end = clock();

quick_time = (double)(end - start) / CLOCKS_PER_SEC;

printf("\nExecution Time for 100 repetitions\n\n");

printf("Bubble Sort   = %f seconds\n", bubble_time);
printf("Selection Sort = %f seconds\n", selection_time);
printf("Insertion Sort = %f seconds\n", insertion_time);
printf("Merge Sort    = %f seconds\n", merge_time);
printf("Quick Sort   = %f seconds\n", quick_time);

return 0;

```

Output

```
Execution Time for 100 repetitions
```

```

Bubble Sort    = 0.127955 seconds
Selection Sort = 0.072278 seconds
Insertion Sort = 0.061300 seconds
Merge Sort     = 0.006278 seconds
Quick Sort     = 0.003966 seconds

```

```
}
```

Bar Chart

